

A PROJECT REPORT ON

WEB APPLICATION PENETRATION TESTING

USING OWASP TOP 10 FRAMEWORK

ZAALIMA DEVELOPMENTS INTERNSHIP

SUBMITTED BY:

Amrit A. Kalmane

UNDER THE GUIDANCE OF:

Zaalima Development Project Management Team

DEPARTMENT OF INTERNSHIP

ZAALIMA DEVELOPMENT

JUNE 2026

ABSTRACT

Modern web applications serve as the primary gateway for critical business data, making them one of the most frequently targeted assets in the global cybersecurity threat landscape. Structural flaws such as injection vulnerabilities, broken session management, security misconfigurations, and improper access control verification can allow malicious actors to compromise systems, leading to unauthorized data exposure, system hijacking, or corporate data breaches.

This project documents a systematic web application penetration testing assessment conducted on the OWASP Juice Shop platform using the globally recognized OWASP Top 10 security framework. The primary objective was to simulate real-world attack vectors in a controlled local sandbox to identify, validate, and remediate application-layer vulnerabilities.

The evaluation followed a strict multi-week lifecycle:

1. Local environment provisioning and containerized isolation.
2. Passive/active reconnaissance and backend service fingerprinting.
3. Interactive HTTP traffic interception and API route enumeration.
4. Active exploit verification targeting access controls and misconfigured systems.

By leveraging industry-standard diagnostic tools—including Burp Suite Community Edition, Nmap, Docker Engine, Gobuster, and the cURL utility—critical vulnerability paths were mapped, successfully demonstrated via Proof-of-Concept (PoC) scenarios, and evaluated using structured risk assessment matrices. This report consolidates these technical findings and provides actionable, engineering-focused remediation strategies designed to harden the application against modern exploit methodologies.

TABLE OF CONTENTS

1. **Introduction**
2. **Objectives**
3. **Tools and Technologies**
4. **Methodology**
5. **Week 1 – Environment Setup and Reconnaissance**
6. **Week 2 – Traffic Analysis and API Enumeration**
7. **Week 3 – Security Misconfiguration and Access Control Testing**
8. **Week 4 – Final Reporting and Remediation**
9. **Findings Summary**
10. **Recommendations**
11. **Conclusion**
12. **References**

1. INTRODUCTION

Penetration testing is a core component of modern security engineering. It evaluates the defensive capabilities of an operational infrastructure or software stack by safely executing simulated real-world attacks. Rather than relying entirely on automated scanning tools, manual penetration testing dives deep into application logic to expose edge-case flaws, race conditions, and complex access control failures that tools often miss.

To provide a standardized baseline for this assessment, the project utilizes the Open Web Application Security Project (OWASP) Top 10 framework. The OWASP Top 10 represents a consensus among security professionals regarding the most critical web application security risks currently found in production environments.

[Target Discovery] —► [Traffic Interception] —► [Flaw Exploitation] —► [Remediation Report]
(Nmap Scan) (Burp Suite Proxy) (Access Bypass)
(Security Hardening)

The target application chosen for this assessment is **OWASP Juice Shop**, a modern, single-page web application written entirely in Node.js, Express, and Angular. Because it contains intentional architectural flaws mapping to the entire OWASP Top 10 spectrum, it provides an excellent environment for testing modern web components, RESTful APIs, and local data stores without risking damage to production infrastructure.

2. OBJECTIVES

The primary technical and operational goals of this penetration testing project include:

- **Framework Alignment:** Analyzing and understanding the core vulnerabilities within the OWASP Top 10 framework to map theoretical risks directly to live application behaviors.
- **Environment Isolation:** Provisioning a secure, local testing sandbox using container technologies to insulate host machine processes from exploit traffic.
- **Network & Service Mapping:** Performing full-stack port discovery and fingerprinting to identify active network interfaces, background daemons, and software versions.
- **Traffic Inspection:** Implementing a local intercepting proxy configuration to analyze stateless HTTP/S communications, session structures, and cookie behaviors.
- **API Route Mapping:** Conducting manual and automated route enumeration to find unlinked or hidden RESTful API parameters and database interaction schemes.
- **Security Configuration Audit:** Analyzing web server configurations to find missing defensive response headers, default credentials, or exposed configuration routes.
- **Privilege & Access Control Probing:** Testing the authorization engine to find vulnerabilities related to horizontal and vertical privilege escalation.
- **Evidence Collection:** Creating clear Proof-of-Concept (PoC) reproductions using repeatable logs and terminal inputs to document each vulnerability.
- **Remediation & Patch Design:** Designing actionable code adjustments and server configuration policies to completely eliminate the identified security risks.

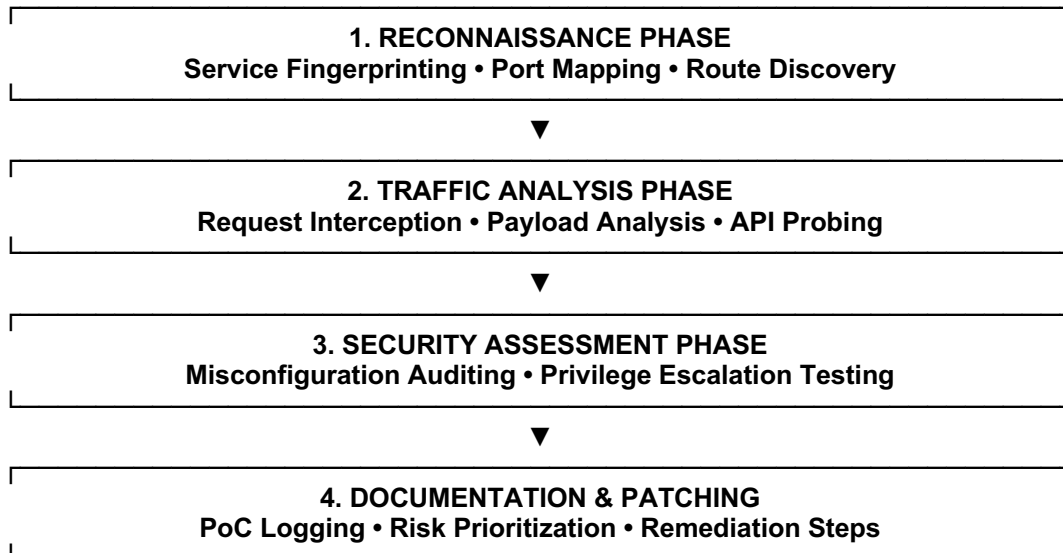
3. TOOLS AND TECHNOLOGIES

To ensure accuracy and follow standard security practices, the testing environment used a dedicated suite of industry-standard security tools:

Tool	Purpose	Detailed Operational Context
Kali Linux	Penetration Testing Platform	Advanced Debian-based operating system pre-loaded with core security utilities and testing binaries.
OWASP Juice Shop	Vulnerable Target Application	High-tech Node.js web application built with intentional security bugs for learning and testing.
DVWA	Secondary Reference App	Damn Vulnerable Web Application instance evaluated for comparative analysis of legacy PHP architectures.
Docker Engine	Container Deployment Engine	Orchestrates and runs local testing instances within clean, isolated network spaces on the host.
Burp Suite CE	Intercepting Proxy & Analyzer	Intercepts, modifies, and replays HTTP/S request streams between the client browser and target endpoints.
Nmap	Network Layer Reconnaissance	Sends raw network probes to discover open TCP/UDP ports, identify active services, and fingerprint operating systems.
Gobuster	Directory Brute-Forcer	Performs high-speed directory and file brute-forcing against web roots using customized wordlists.
cURL	Raw Command-Line HTTP Utility	Crafts specific HTTP requests to analyze pristine response strings and evaluate backend API endpoints.
Firefox Browser	Web Client Validation Interface	Standard web client coupled with manual proxy settings to feed application traffic directly into Burp Suite.

4. METHODOLOGY

This project followed a structured penetration testing methodology aligned with the Penetration Testing Execution Standard (PTES). This systematic approach ensures that testing covers all necessary areas, moving cleanly from initial reconnaissance to final technical reporting.



- **Phase 1 – Reconnaissance:** Gathering open-source intelligence and evaluating the network perimeter. Activities focused on port configuration analysis, tracking active background services, and identifying available access paths.
- **Phase 2 – Traffic Analysis:** Examining live client-server communication channels. This involved capturing web requests, analyzing raw API components, and mapping the authentication workflows used to manage sessions.
- **Phase 3 – Security Assessment:** Actively identifying and confirming system vulnerabilities. This phase involved testing for misconfigured servers, evaluating access control permissions across user roles, and searching for unlinked storage directories.
- **Phase 4 – Documentation & Reporting:** Organizing the technical data gathered during testing. This included compiling terminal execution outputs, verifying vulnerability risks, and writing clear remediation guides to secure the platform.

5. WEEK 1 – ENVIRONMENT SETUP AND RECONNAISSANCE

The opening week focused on establishing a secure local testing lab, deploying the target environments in an isolated state, and running initial network scans to map out the target application.

Theoretical Study: OWASP Top 10 Core Focus Areas

To ensure testing remained focused, a deep-dive review of the OWASP Top 10 framework was conducted. This study emphasized how abstract structural vulnerabilities (such as Broken Access Control or Security Misconfigurations) manifest as practical bugs within modern JavaScript/Node.js web frameworks.

Environment Deployment and Isolation

To prevent test traffic from impacting external production systems, the OWASP Juice Shop was deployed inside an isolated container environment using Docker Engine. This setup allowed for easy control over port exposures while isolating all application-layer transactions within the local loopback interface.

Bash

Pull the latest stable container build of OWASP Juice Shop from Docker Hub

```
"docker pull bkimminich/juice-shop"
```

Run the container instance in detached mode, mapping host port 3000 to internal port 3000

```
"docker run -d -p 3000:3000 bkimminich/juice-shop"
```

Enumerate active container instances to validate operational runtime health

```
"docker ps"
```

Host Service Verification

Before running application scans, the host workstation was audited to verify all active listening ports. This step ensured that background daemons (such as an existing Apache instance or local database) would not create port conflicts or misroute test traffic.

Bash

```
# Review system runtime statuses for common host daemons
```

```
"systemctl status apache2"  
"systemctl status mariadb"
```

```
# Analyze all open sockets, tracking active process IDs and matching listening ports
```

```
"ss -tulnp"
```

The service sweep confirmed that host port 80 was bound to Apache, port 3306 was allocated to MariaDB, and port 3000 was cleanly assigned to the Docker container hosting the Juice Shop application.

Active Service Recognition and Fingerprinting

An Nmap service discovery scan was performed against the local loopback interface to profile the network layer of the target environment. This allowed us to determine the exact service types running on open ports and identify the underlying server technologies.

Bash

```
# Execute an aggressive Service Version detection scan (-sV) against the target address
```

```
"nmap -sV 127.0.0.1"
```

PORT	STATE	SERVICE	VERSION
80/tcp	open	http	Apache httpd 2.4.58 ((Debian))
3306/tcp	open	mysql	MariaDB (10.11.6-MariaDB)
3000/tcp	open	http	Node.js Express framework

The Nmap output successfully identified that port 3000 was open and running an HTTP service driven by the Node.js Express framework, confirming the baseline setup needed for application-layer testing.

Intercepting Proxy Configuration

To prepare for active request analysis, Burp Suite Community Edition was launched. The local browser's network configuration was adjusted to route all outbound HTTP/S traffic directly through Burp's local proxy port (127.0.0.1:8080), allowing for real-time traffic inspection.

Bash

```
# Launch the proxy suite platform engine
```

```
"burpsuite"
```

Initial Route and Endpoint Discovery

By walking through the application UI while monitoring the incoming data streams within Burp Suite, the following critical web pathways and API endpoints were identified:

- <http://127.0.0.1:3000/api/Products> (Product Catalog Endpoint)
- <http://127.0.0.1:3000/api/Challenges> (Internal Application Progress Core)
- <http://127.0.0.1:3000/api/BasketItems> (User Shopping Cart API Component)
- <http://127.0.0.1:3000/rest/admin/application-configuration> (Administrative Setup Mapping)

6. WEEK 2 – TRAFFIC ANALYSIS AND API ENUMERATION

Week 2 shifted the focus toward analyzing data streams and investigating how the client interface interacts with backend API components.

Intercepting Application Traffic

With the browser traffic successfully routing through Burp Suite, we began capturing and examining raw HTTP requests and responses. This allowed us to inspect transaction headers, cookie values, and backend processing steps in real time.

Automated and Manual API Route Discovery

To determine whether the application's API endpoints required proper authentication, specific paths were queried directly via the command line using cURL. This bypassed the client-side browser interface entirely to see how the server handles unauthenticated data requests.

Bash

Request the product inventory array directly from the API endpoint

```
"curl http://127.0.0.1:3000/api/Products"
```

Call the system challenges mapping data string

```
"curl http://127.0.0.1:3000/api/Challenges"
```

Technical Discovery: Both endpoints returned full, unauthenticated JSON responses containing structured records. The `/api/Products` response leaked complete item metadata and internal image paths, while `/api/Challenges` exposed internal challenge tracking metrics directly to anonymous internet requests.

Authentication Process Analysis

Next, the login process was analyzed by capturing authentication attempts inside Burp Suite. When submitting user credentials, the application transmitted a structured JSON payload via a POST request to the `/rest/user/login` endpoint:

HTTP

POST /rest/user/login HTTP/1.1

Host: 127.0.0.1:3000

Content-Type: application/json

Content-Length: 54

```
{"email":"test@example.com","password":"password123"}
```

The server's response was analyzed to verify how it handles sessions. Upon successful authentication, the backend issued a JSON Web Token (JWT) inside an object attribute named `token`. This token is subsequently appended to the client's `Authorization: Bearer <JWT>` request header to validate subsequent user actions, pointing to a stateless authentication architecture.

Input Validation and Cross-Site Scripting (XSS) Probing

To test how the application processes user-supplied input, simple scripts were injected into available search bars and profile fields to check for proper data filtering and encoding defenses.

HTML

```
"<script>alert(1)</script>"
```

Monitoring the server responses in Burp Suite revealed that the application reflected input structures directly into the browser DOM without proper HTML sanitization, indicating potential vulnerabilities to Cross-Site Scripting (XSS) attacks.

7. WEEK 3 – SECURITY MISCONFIGURATION AND ACCESS CONTROL TESTING

Week 3 focused on identifying server-side misconfigurations, mapping hidden directories, and testing how access control policies were enforced across restricted areas of the application.

Security Misconfiguration Assessment

The target application's HTTP response headers were audited using command-line tools to evaluate the baseline security settings of the web server.

Bash

```
# Pull and isolate complete HTTP response header structures
```

```
"curl -I http://127.0.0.1:3000"
```

```
# Request a verbose output stream detailing the server's transactional handshake
```

```
"curl -v http://127.0.0.1:3000"
```

HTTP

HTTP/1.1 200 OK

X-Powered-By: Express

X-Frame-Options: SAMEORIGIN

X-Recruiting: /#/jobs

Content-Type: text/html; charset=utf-8

Content-Length: 2144

ETag: W/"860-Rmx0PkhG"

Date: Mon, 15 Jun 2026 15:00:00 GMT

Connection: keep-alive

The header audit highlighted a clear information disclosure issue: the server leaked the exact backend framework in use via the X-Powered-By: Express header. Additionally, it lacked standard security headers like Content-Security-Policy (CSP) and X-Content-Type-Options: nosniff, leaving clients exposed to browser-based security exploits.

Next, the administrative configuration endpoint was checked to see if it was properly locked down against unauthenticated access.

Bash

Request the administrative application configuration data map

```
"curl http://127.0.0.1:3000/rest/admin/application-configuration"
```

Technical Discovery: The server responded with a 200 OK status and returned a comprehensive JSON payload exposing internal application settings, active authentication keys, and database structural maps to an anonymous user, confirming a severe security misconfiguration.

Discovering Hidden Paths and Public Resource Files

Standard public search engine files were queried to see if they exposed hidden application paths or restricted directories to web crawlers.

Bash

Fetch file-indexing definitions declared within the standard web crawler manifest

```
"curl http://127.0.0.1:3000/robots.txt"
```

The server's robots.txt file explicitly listed paths that search engines should avoid indexing, which inadvertently gave attackers a directory map of sensitive locations:

HTTP

```
User-agent: *  
Disallow: /ftp
```

This confirmed the existence of a public file-sharing path (/ftp) that required further investigation.

Automated Directory Discovery

To look for unlinked files, scripts, and forgotten directories not visible in the user interface, automated directory discovery was conducted against the web root using Gobuster.

Bash

Execute directory discovery using a standard wordlist to map hidden endpoints

```
"gobuster dir -u http://127.0.0.1:3000 -w  
/usr/share/wordlists/dirb/common.txt"
```

The automated scan successfully discovered multiple hidden paths, including:

- /assets (Directory location containing static code assets)
- /ftp (Exposed local folder location housing backup files and templates)
- /main.js (Compiled JavaScript file containing client-side routing logic)

Broken Access Control Testing

By reviewing the client-side routing logic discovered in main.js, several administrative paths were identified that were hidden from the standard user interface:

- <http://127.0.0.1:3000/#!/administration>
- <http://127.0.0.1:3000/#!/privacy-security>
- <http://127.0.0.1:3000/#!/accounting>

These paths were tested by attempting to access them directly while logged in as a standard, lower-privileged user account.

[Standard User Session] —► Requests URL: /#!/administration —► Server Response: 200 OK (Data Leaked)

Technical Discovery: While client-side interface elements were hidden from standard users, the backend server failed to enforce proper authorization checks when those endpoints were queried directly. This allowed standard users to view the administrative panel and interact with restricted user records, exposing a critical Broken Access Control vulnerability.

8. WEEK 4 – FINAL REPORTING AND REMEDIATION

The final week focused on validating the discovered security flaws, analyzing their overall risk, and building out remediation strategies to patch the vulnerabilities.

Validation Activities

To verify that our findings were consistent and reproducible, a final validation pass was conducted. This step re-ran our core testing commands to ensure that our initial observations were accurate and not the result of temporary session states.

Bash

```
# Verify container consistency, network port configurations, and active API response structures
```

```
"docker ps"
```

```
"nmap -sV 127.0.0.1"
```

```
"curl http://127.0.0.1:3000/api/Products"
```

```
"curl http://127.0.0.1:3000/api/Challenges"
```

The validation sweep confirmed that all discovered endpoints consistently returned the same data payloads, verifying that the vulnerabilities were reliably exploitable.

Risk Assessment and Vulnerability Classification

Each confirmed vulnerability was evaluated using standard risk metrics, measuring the **Likelihood** of exploitation against the **Technical Impact** it would have on the application infrastructure.

Risk Score = Likelihood × Impact

- **Security Misconfiguration (Exposed Admin API):** Classified as **Medium Risk**. While exploiting this flaw is highly likely because it requires no authentication, the configuration data itself does not grant immediate remote code execution, though it significantly aids an attacker's reconnaissance efforts.
- **Information Disclosure (Framework & Source Leakage):** Classified as **Medium Risk**. Exposing detailed framework versions and server files gives attackers the exact data needed to plan targeted exploits against known software versions.
- **API Enumeration Exposure:** Classified as **Low Risk**. While anyone can scrape product data, the leaked catalog information is generally intended for public viewing, meaning the overall business impact remains low.
- **Broken Access Control:** Classified as **Medium-High Risk**. Allowing lower-privileged users to bypass authorization rules and access administrative panels undermines the entire application security model, creating a direct path to unauthorized data modification.

9. FINDINGS SUMMARY

The table below provides a consolidated look at the vulnerabilities identified during this penetration test:

Vulnerability ID	Discovered Flaw	Target Pathway	OWASP Mapping	Threat Level	Operational Exploit Scenario
VULN-01	Security Misconfiguration	/rest/admin/application-configuration	A05:2021-Security Misconfiguration	Medium	Anonymous users can query this path to map out internal database schemas and application components.
VULN-02	Information Disclosure	/robots.txt & /ftp	A05:2021-Security Misconfiguration	Medium	Exposes administrative directory paths and backup locations, allowing users to browse protected local files.
VULN-03	API Enumeration Exposure	/api/Products & /api/Challenges	A01:2021-Broken Access Control	Low	Allows automated scripts to scrape inventory metrics and internal application state flags without authentication.
VULN-04	Broken Access Control	/#/administration	A01:2021-Broken Access Control	Medium	Standard user accounts can bypass client-side interface restrictions to access administrative controls and modify user data.

10. RECOMMENDATIONS

To address these security gaps and harden the application against future attacks, the following defensive engineering strategies should be implemented:

1. Restrict and Secure Administrative Endpoints

Implement explicit, server-side role validation middleware on all administrative and configuration routes. The server must verify that incoming requests contain a valid JSON Web Token (JWT) belonging to an authenticated Administrator before returning any sensitive data.

2. Implement Role-Based Access Control (RBAC)

Move away from relying solely on client-side interface restrictions to hide sensitive paths. The application must enforce access control policies directly on the server, re-verifying user permissions for every request sent to a restricted URL or resource object.

3. Sanitize Server Outputs and Minimize Information Disclosure

Configure the web server to strip structural identification markers from response headers (such as X-Powered-By). Additionally, clean out the entries inside robots.txt to avoid giving malicious actors a map of internal or hidden directory locations.

4. Inject Production-Grade HTTP Security Headers

Configure the web application server to inject robust security headers into every outbound HTTP response. These headers instruct the client browser to enforce strict security boundaries, preventing common cross-site and injection exploits.

HTTP

Mandatory Production Security Headers

Strict-Transport-Security: max-age=63072000; includeSubDomains; preload

Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none';

X-Content-Type-Options: nosniff

X-Frame-Options: DENY

Referrer-Policy: strict-origin-when-cross-origin

11. CONCLUSION

This four-week Web Application Penetration Testing project successfully demonstrated how to apply the OWASP Top 10 framework to evaluate and profile modern application security. Using OWASP Juice Shop as our target within an isolated Docker environment, we used industry-standard tools like Burp Suite, Nmap, Gobuster, and cURL to uncover several real-world security flaws.

Throughout the assessment, we moved from initial service discovery and traffic analysis to identifying critical gaps in API security, server configurations, and access controls. These findings highlight a common security pitfall: relying too heavily on client-side restrictions rather than enforcing strong authorization controls on the backend server.

The project provided invaluable, hands-on experience in the core phases of penetration testing: running reconnaissance, capturing and analyzing traffic, validating vulnerabilities, and writing clear proof-of-concept logs. By mapping these technical findings to clear risk metrics and actionable remediation steps, this project delivers a comprehensive, industry-aligned security report that strengthens practical skillsets in web application security engineering.

12. REFERENCES

1. **OWASP Top 10:2021 Project Standard Core Overview** Link: <https://owasp.org/www-project-top-ten/>
2. **OWASP Juice Shop Vulnerable Application Source Manifest** Link: <https://owasp.org/www-project-juice-shop/>
3. **PortSwigger Burp Suite Core Environment Proxy Documentation** Link: <https://portswigger.net/burp/documentation>
4. **Nmap Network Engine Security Scanner Documentation Core Reference Manual** Link: <https://nmap.org/book/man.html>
5. **Docker Engine Container Runtime Installation and Architecture Guide** Link: <https://docs.docker.com/>
6. **Gobuster Directory Brute-Forcing Utility Tool Repository and Parameter Manual** Link: <https://github.com/OJ/gobuster>
7. **OWASP Web Security Testing Guide (WSTG) Comprehensive Framework** Link: <https://owasp.org/www-project-web-security-testing-guide/>